蓝桥杯 Python 进阶全景技能大纲

1. 命名规范约定

抛弃随意命名，建立你的肌肉记忆：

- **数组/列表**：`arr`, `nums` (一般数组), `prefix` (前缀和数组), `diff` (差分数组)。
- **哈希表/字典**：`freq_map` (频次统计), `pos_dict` (记录位置), `memo` (记忆化搜索字典)。
- **指针/索引**：`left`, `right` (双指针), `slow`, `fast` (快慢指针), `idx` (通用索引)。
- **结果与最值**：`ans`, `res` (最终结果), `max_val`, `min_val`。
- **图论相关**：`adj` (邻接表), `vis` / `visited` (访问标记集合)。

2. 标准输入输出范式 (I/O Template)

Python 在处理大规模数据

$$(\text{如}10^5\text{级别以上})\text{时内置的}\backslash input()\backslash\text{会成为性能瓶颈。必须使用}sys\text{模块的快速}I/O\text{。}$$

```
import sys
# 针对递归深度报错的防御性设置 (蓝桥杯必加)
sys.setrecursionlimit(200000)

def solve():
    # 快速读取所有内容并按空白字符分割
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # 示例：读取 n 和 m
    n = int(input_data[0])
    m = int(input_data[1])

    # 示例：读取一个长度为 n 的数组
    arr = [int(x) for x in input_data[2:2+n]]

    # 你的核心逻辑...
    ans = 0

    # 标准输出
    print(ans)

if __name__ == '__main__':
    solve()
```

---

3. 终极解题心法：如何面对任何一道题

在没有 OJ 实时判题的情况下，你的大脑就是编译器。

1. **数据范围倒推时间复杂度（极其关键）**：
   - $$\text{看到}N \le 20\text{：考虑回溯、}DFS\text{、状态压缩}DP(O(2^N)\text{或}O(N!))\text{。}$$
   - $$\text{看到}N \le 1000\text{：考虑二维}DP\text{、多重循环}(O(N^2))\text{。}$$
   - $$\text{看到}N \le 10^5\text{：算法必须是}O(N)\text{或}O(N\log N)\text{。考虑贪心、双指针、单调栈/队列、线段树、二分答案。}$$

- 看到$N \le 10^9$：考虑数学规律、数论、矩阵快速幂($O(\log N)$或$O(1)$)。

2. **“先暴力，再优化”策略**：
   - 蓝桥杯按测试点给分。拿到题毫无思路时，**一定要先写一个最暴力的解法（如多重循环、全排列DFS）**，这能稳拿 20%-30% 的分数。
   - 在暴力解的基础上，观察重复计算的部分，这通常是引入哈希表、前缀和或动态规划的切入点。
3. **极端数据测试（哨兵思维）**：
   - 手动测试：N=0，N=1，数组全为正数、全为负数、全部相同的情况。
   - **哨兵技巧（Sentinel）**：在处理数组边界（如滑动窗口、链表、栈）时，在首尾添加虚拟元素（如极大值 `float('inf')` 或虚拟节点），可以免去大量繁琐的 `if` 边界判断。

4.核心知识点与数据结构详表

模块类别	核心知识点与技能要求	蓝桥杯考频与重要度
0. Python 竞技特性	快速 I/O（`sys.stdin.read`）； 递归深度解除（`sys.setrecursionlimit`）； 内置库压榨（`collections`, `itertools`, `bisect`, `heapq`）； 记忆化搜索（`@lru_cache`）。	⭐⭐⭐⭐⭐ (基石)
1. 核心思维技巧	双指针（对撞、快慢）； 滑动窗口（定长、不定长）； 前缀和（一维、二维）； 差分数组（区间加减）； 离散化（值域过大时的坐标映射）； 哨兵技巧（边界防御）； 位运算。	⭐⭐⭐⭐⭐ (必考)
2. 高级数据结构	栈和队列/单调栈和单调队列（寻找最近极值、滑动窗口最值）； 并查集（连通性问题、路径压缩）； 字典树 Trie（字符串前缀匹配）； 线段树/树状数组（高级区间求和/修改，省赛拔高题，国赛常客）。	⭐⭐⭐⭐ (分水岭)
3. 搜索与图论	DFS & 回溯（排列组合、子集、剪枝策略）； BFS（最短路、多源 BFS）； 拓扑排序（依赖关系）； 最短路径（Dijkstra, Floyd）； 最小生成树（Kruskal, Prim）； 并查集(DSU)。	⭐⭐⭐⭐⭐ (大题票仓)
4. 动态规划 (DP)	线性 DP（最长递增子序列 LIS，最长公共子序列 LCS）； 背包九讲（01背包、完全背包、多重背包及空间优化）； 区间 DP（石子合并）； 树形 DP（打家劫舍III）； 状态压缩 DP（哈密顿路径，$N \le 20$ 极客解法）。	⭐⭐⭐⭐⭐ (王冠上的明珠)
5. 贪心算法	区间调度问题（贪心策略证明）； 微观操作求最值（排序+优先队列）； Huffman 树思想。	⭐⭐⭐⭐ (极易错)
6. 数学与数论	最大公约数 GCD（辗转相除）、 最小公倍数 LCM； 快速幂（取模运算）； 素数筛法（埃氏筛、欧拉线性筛）； 组合数学（排列组合取模、乘法逆元）； 容斥原理。	⭐⭐⭐ (防 AK 题)
7. 递归	递归逻辑设计、递归终止条件（避免栈溢出）； 结合记忆化搜索优化重复计算，常用于排列组合、树形问题。	⭐⭐⭐⭐ (基础必备)
8. 分治	分而治之思想，将复杂问题拆解为若干子问题，求解子问题后合并结果； 典型应用：归并排序、快速排序、大整数乘法。	⭐⭐⭐ (重点应用)
9. 二分查找	适用于有序序列的查找、边界值求解（如“最大值最小化”问题）； 核心是确定二分区间和判断条件，避免边界遗漏。	⭐⭐⭐⭐ (必考)
10. 快速排序	基于分治思想，选择基准值划分区间，高效排序； 时间复杂度 O(n\log n)，蓝桥杯中常用于快速处理数据排序需求。	⭐⭐⭐ (基础排序)
11. 归并排序	基于分治思想，先拆分后合并，稳定排序； 时间复杂度 O(n\log n)，常用于需要稳定排序或求逆序对的场景。	⭐⭐⭐ (重点排序)

模块类别	核心知识点与技能要求	蓝桥杯考频与重要度
12. 冒泡排序	简单排序算法，通过相邻元素交换实现排序； 时间复杂度 O(n^2)，仅适用于数据量极小的场景（蓝桥杯偶尔用于小题快速编写）。	⭐ (基础了解)
13. 桶排序/计数排序	非比较类排序，适用于数据范围明确、离散且范围不大的场景； 计数排序针对整数，桶排序可处理更广泛数据，效率极高（$O(n)$）。	⭐ ⭐ (场景适配)
14. 字符串	KMP 算法（字符串匹配，高效解决子串查找）； 字符串哈希（将字符串映射为数值，快速判断字符串相等）； 字典树 Trie（字符串前缀匹配、词频统计）。	⭐ ⭐ ⭐ ⭐ (高频考点)

5.蓝桥杯实战解题口诀（必背）

口诀对应实战场景，快速选择解题思路，高效突破时间限制，适配蓝桥杯不同难度题目：

```
数据小直接暴力
数据大急需优化
能证明就上贪心
不能证明先暴搜
暴搜不过就剪枝
剪枝超时上DP
DP爆了压状态
再不行优化输入
打表退火加随机
无序查找哈希表
有序查找平衡树
区间求和前缀和
有序问题上二分
最值问题搞个堆
```

6.讲解

为了让你直观感受这些“Python 竞技特性”的威力，我分别为每个知识点匹配了经典的蓝桥杯风格题目，并配以核心逻辑解析。

1.1快速 I/O (`sys.stdin.read`)

> 场景：当题目数据量达到10^5到10^6级别时，普通的\$input()\$会因为频繁的系统调用导致超时（TLE）。

🚩 匹配题目：区间求和（基础版）

> 题目描述：给定一个长度为N的数组，进行M次查询，每次查询区间$[L, R]$的和。$N, M \leq 2 \times 10^5$。

【讲解】：

在 Python 中，`input()` 每次只读取一行。而 `sys.stdin.read().split()` 会一次性将整个标准输入读入内存，并按空格/换行切分成列表。这在处理数十万个整数输入时，速度提升通常在 **5-10 倍**。

```
import sys

def solve():
    # 一次性读取所有数据
    data = sys.stdin.read().split()
    if not data: return

    n = int(data[0])
    m = int(data[1])
```

```
    # 构建前缀和
    arr = list(map(int, data[2:2+n]))
    s = [0] * (n + 1)
    for i in range(n):
        s[i+1] = s[i] + arr[i]

    # 处理查询
    results = []
    ptr = 2 + n
    for _ in range(m):
        l = int(data[ptr])
        r = int(data[ptr+1])
        results.append(str(s[r] - s[l-1]))
        ptr += 2

    # 一次性输出，避免多次调用 print
    sys.stdout.write("\n".join(results) + "\n")

# solve()
```

1.2 递归深度解除（`sys.setrecursionlimit`）

场景：$Python$默认递归深度仅1000。在处理深层DFS或树形结构（如$N = 10^5$的树）时，直接运行会抛出\$RecursionError\$。

🚩 匹配题目：树的深度/连通分量

> 题目描述：给定一棵N个节点的树，求根节点到最远叶子节点的距离。$N = 10^5$。

【讲解】：

蓝桥杯的省赛题目中，树和图的遍历非常多。如果不用 `sys.setrecursionlimit`，你的 DFS 只要走过 1000 层就会报错。

```
import sys
# 必须手动设置，建议设为 10^5 或 2*10^5
sys.setrecursionlimit(200000)

def dfs(u, p, depth, adj):
    max_d = depth
    for v in adj[u]:
        if v != p:
            max_d = max(max_d, dfs(v, u, depth + 1, adj))
    return max_d
```

1.3 内置库压榨（`collections`，`itertools`，`bisect`，`heapq`）

场景： Python 运行慢，但其 C 语言实现的内置库极快。学会“调包”是抢时间的关键。

🚩 匹配题目：超市购物最优策略

> **题目描述**：有 N件商品，每件有过期时间和利润。每天只能卖一件，求最大利润。

【讲解】：

- `heapq`：用于维护动态最值（小顶堆）。
- `collections.deque`：实现 $O(1)$ 的双端队列（BFS 必用）。
- `bisect`：在有序序列中进行二分查找。
- `itertools`：快速生成全排列 `permutations` 或组合 `combinations`（暴力搜索神器）。

```
import heapq

# 假设商品按过期时间排序，用堆维护当前已选商品的利润
# 如果新商品利润更高但时间紧，可以用它替换掉堆顶（利润最低）的商品
profit_heap = []
for p, t in items:
    if t > len(profit_heap):
        heapq.heappush(profit_heap, p)
    elif profit_heap and p > profit_heap[0]:
        heapq.heapreplace(profit_heap, p)
```

1.4 记忆化搜索（`@lru_cache`）

场景： 将暴搜（DFS）直接转化为动态规划（DP），避免重复计算。

🚩 匹配题目：数字三角形 / 爬楼梯进阶版

> **题目描述**：在一个迷宫中，从起点到终点有多少种走法？每一步有多种选择，且存在大量重复状态。

【讲解】：

这是 Python 选手的“降维打击”武器。你只需要写一个逻辑正确的 DFS，然后在函数头加上一行装饰器，它就会自动开辟哈希表记录结果。

```
from functools import lru_cache

@lru_cache(None) # None 表示不限制缓存大小
def dfs(x, y):
    if x == target_x and y == target_y:
        return 1

    res = 0
    # 假设只能向右或向下
    for dx, dy in [(0, 1), (1, 0)]:
        nx, ny = x + dx, y + dy
        if check(nx, ny):
            res += dfs(nx, ny)
    return res

# 只要状态组合（x, y）出现过，第二次调用直接返回缓存结果，时间复杂度瞬间从指数级降为 O(N*M)
```

📝 总结建议

快速I/O题目——输入整数超过5×10^4个时。🚀🚀🚀🚀

递归解除——涉及树、深层DFS搜索时（必写）。🛠（防崩溃）

内置库——只要能用$\backslash collections\backslash,\backslash heapq\backslash$就不用手写。🚀🚀🚀

记忆化——DFS出现重复状态，且不想手推DP转移方程时。🚀🚀🚀🚀🚀

2.1 对撞指针：两数之和（有序版）

场景：利用单调性收缩搜索空间。

双指针：对撞指针 (Two Pointers)

例题： [两数之和 II - 输入有序数组]

> **描述**：给定一个升序排列的整数数组 `numbers`，从数组中找出两个数，使它们的和等于目标数 `target`。

- **传统思维**：暴力双层循环，O(N^2)。
- **双指针思维**：利用**有序性**。左指针 L 指向头，右指针 R 指向尾。

- $-$如果$Sum > target$，说明太大了，R向左移动。
- $-$如果$Sum < target$，说明太小了，L向右移动。

- **复杂度**：O(N)。

```
def two_sum(nums, target):
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target:
            return [l + 1, r + 1] # 返回下标
        elif s < target:
            l += 1 # 太小了，左指针右移
        else:
            r -= 1 # 太大了，右指针左移
    return []
```

2.2 滑动窗口：长度最小的子数组

场景：处理“连续区间”的最值问题。

滑动窗口：不定长窗口 (Sliding Window)

例题： [长度最小的子数组]

> 描述：给定一个含有n个正整数的数组和一个正整数$target$。找出该数组中满足其和 $\geq target$的长度最小的连续子数组。

- **技巧点**：窗口的“伸缩”。
  1. **伸**：右指针 `r` 向右移，将元素加入当前和 `cur_sum`。
  2. **缩**：当 `cur_sum >= target` 时，尝试缩小左边界 `l` 来更新最小长度。
- **代码片段**：
-
  ```
  l, cur_sum, res = 0, 0, float('inf')
  for r in range(len(nums)):
      cur_sum += nums[r]
      while cur_sum >= target:
          res = min(res, r - l + 1)
          cur_sum -= nums[l] # 缩
          l += 1
  ```

```
def min_sub_array(target, nums):
    l = 0
    cur_sum = 0
    res = float('inf')
    for r in range(len(nums)):
        cur_sum += nums[r]
        # 当窗口内和达标时，尝试收缩左边界
        while cur_sum >= target:
            res = min(res, r - l + 1)
            cur_sum -= nums[l]
            l += 1
    return res if res != float('inf') else 0
```

2.3 一维前缀和：区域和检索

场景：将$O(N)$的区间查询降为$O(1)$。

前缀和与差分 (Prefix Sum & Difference)

例题（差分）：[航班预订统计 / 区间加法]

> 描述：数组初始全为0。有M个操作：在区间$[L, R]$内的所有数都加上V。求最终数组。

- 传统思维：每次循环修改，$O(M \times N)$，数据量大必超时。
- **差分思维**：
  - 构建差分数组D。区间修改变点修改：$D[L] += V$，$D[R+1] -= V$。
  - 最后对 D 求一遍前缀和，还原出原数组。
- **复杂度**：O(M + N)。

```
import sys

def prefix_sum_demo():
    # 模拟快速读取
    data = sys.stdin.read().split()
    n, m = int(data[0]), int(data[1])
    nums = list(map(int, data[2:2+n]))

    # 1. 构建前缀和数组 (多开一位处理边界)
    P = [0] * (n + 1)
    for i in range(n):
        P[i+1] = P[i] + nums[i]

    # 2. O(1) 查询区间 [l, r] 的和 (l, r 为 1-indexed)
    # 公式：P[r] - P[l-1]
```

2.4 差分数组：增量修改

场景：批量区间加减，最后统一结算。

二维前缀和 (2D Prefix Sum)

例题：[最大子矩阵]

> 描述：给定一个$N \times N$的矩阵，求其中和最大的子矩阵。

- **技巧点**：二维积分图。
  - $S[i][j]$表示从$(0, 0)$到(i, j)矩形内所有元素的和。
  - 任意子矩形$[x1, y1]$到$[x2, y2]$的和为：

$$S[x2][y2] - S[x1-1][y2] - S[x2][y1-1] + S[x1-1][y1-1]$$

- **意义**：将 $O(N^2)$ 的区域求和降到 $O(1)$。

```
def difference_array(n, updates):
    # n: 数组长度, updates: [[l, r, val], ...]
    diff = [0] * (n + 2)
    for l, r, val in updates:
        diff[l] += val
        diff[r + 1] -= val

    # 还原数组（求前缀和）
    res = [0] * (n + 1)
    for i in range(1, n + 1):
        res[i] = res[i-1] + diff[i]
    return res[1:]
```

2.5 离散化：区间覆盖

$$\text{场景：坐标范围大（}10^9\text{）但点数少（}10^3\text{）。}$$

离散化 (Discretization)

例题：[区间覆盖长度]

> $$\text{描述：给定}10^5\text{条线段，坐标范围为}[-10^9, 10^9]\text{，求这些线段覆盖的总长度。}$$

- **技巧点**：值域极大但点数有限。
  - 将所有出现的起点、终点坐标存入列表，排序并去重。
  - $$\text{原本}20\text{亿的范围被压缩成了}2 \times 10^5\text{个有序的“排名”。}$$
  - 在新坐标轴上利用**差分**或**扫描线**处理。

```
def discretization(intervals):
    # 1. 收集所有坐标并去重排序
    alls = []
    for l, r in intervals:
        alls.append(l)
        alls.append(r)
    alls = sorted(list(set(alls)))

    # 2. 映射函数：查找坐标在新数组中的索引
    def find(x):
        import bisect
        return bisect.bisect_left(alls, x)

    # 之后可以在 find(x) 得到的 [0, 2000] 范围内做差分或线段树
```

2.6 哨兵技巧：单调栈最大矩形

哨兵技巧 (Sentinel)

例题：[柱状图中最大的矩形]

> $$\text{描述：给定}n\text{个非负整数表示每个柱子的高度，求能勾勒出的最大矩形面积。}$$

- **技巧点**：在数组末尾手动添加一个高度为 `0` 的柱子。
- **作用**：在使用单调栈时，如果没有这个 `0` 哨兵，循环结束后还需要额外写代码清空栈内剩余的元素。有了 `0`，所有还没处理的柱子都会因为遇到这个“最小值”而被迫弹出并计算面积。

场景：强制清空栈，简化逻辑。

```
def largest_rectangle(heights):
```

```
    # 加上哨兵 0，确保最后所有柱子都能出栈结算
    heights.append(0)
    stack = [-1]
    max_area = 0

    for i in range(len(heights)):
        # 当前高度小于栈顶高度，触发结算
        while heights[i] < heights[stack[-1]]:
            h = heights[stack.pop()]
            w = i - stack[-1] - 1
            max_area = max(max_area, h * w)
        stack.append(i)
    return max_area
```

2.7 位运算：状态压缩 DP（旅行商简化版）

位运算 (Bit Manipulation)

例题：[汉密尔顿路径 (TSP 问题)]

> 描述：经过N个点各一次的最短路径$N \leq 20$。

- **技巧点**：用一个整数表示"访问过的点集"。
  - 如果第 i 位是 1，代表第 i 个点去过了。
  - `state | (1 << i)`：标记第 i 个点已访问。
  - `state & (1 << i)`：检查第 i 个点是否访问过。
- **意义**：将复杂的集合组合问题转化成极快的位运算。

场景：用一个数字代表集合状态。

```
from functools import lru_cache

def tsp_simple(n, dist):
    # state: 二进制位表示去过的城市, u: 当前所在城市
    @lru_cache(None)
    def dfs(state, u):
        if state == (1 << n) - 1: # 所有城市都去过了
            return dist[u][0] # 返回起点

        res = float('inf')
        for v in range(n):
            # 如果第 v 位为 0，表示没去过
            if not (state & (1 << v)):
                res = min(res, dfs(state | (1 << v), v) + dist[u][v])
        return res

    return dfs(1, 0) # 从城市 0 出发
```

3.1. 单调栈与单调队列 (Monotonic Stack/Queue)

核心点：

- **单调栈**：找元素左/右边**第一个**比它大/小的元素。O(n)。
- **单调队列**：维护动态区间的**最值**。O(n)。

🚩 题目：滑动窗口最大值

> **描述**：给一个数组 `nums` 和窗口大小 `k`，窗口从左向右滑动，输出每个窗口内的最大值。

【代码实现】

```
from collections import deque

def max_sliding_window(nums, k):
    q = deque() # 存储下标，且 nums[q] 保持单调递减
    res = []
    for i in range(len(nums)):
        # 1. 入队前：踢出比当前值小的元素（它们永远没机会成为最大值了）
        while q and nums[q[-1]] <= nums[i]:
            q.pop()
        q.append(i)
        # 2. 检查队头是否过期（超出了窗口范围）
        if q[0] <= i - k:
            q.popleft()
        # 3. 窗口形成后开始记录答案
        if i >= k - 1:
            res.append(nums[q[0]])
    return res
```

3.2. 并查集 (Disjoint Set Union, DSU)

核心点：解决"谁和谁是一伙的"连通性问题。

- **路径压缩**：`find` 时让节点直接指向根，使树高几乎为常数。
- **按秩合并**：小树合并到大树（可选，通常路径压缩就够快了）。

🚩 题目：连通块中点的数量

> 描述：n个点，m次操作：1.连接两点；2.询问两点是否在同一连通块；3.询问某点所在连通块的大小。

【代码实现】

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n + 1))
        self.size = [1] * (n + 1)

    def find(self, x):
        if self.parent[x] != x:
            # 路径压缩核心：递归找根并更新指向
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rootX, rootY = self.find(x), self.find(y)
        if rootX != rootY:
            self.parent[rootX] = rootY
            self.size[rootY] += self.size[rootX]
```

3.3. 字典树 (Trie)

核心点：高效处理字符串前缀。每个节点是一个字符，从根到某节点的路径即为一个前缀。

🚩 题目：前缀统计

> 描述：给定n个单词和m个查询单词，问查询单词作为前缀在字典中出现了几次。

【代码实现】

Python

```
class Trie:
    def __init__(self):
```

```
        self.children = {} # 字典实现，动态省空间
        self.count = 0     # 经过该节点的路径数

    def insert(self, word):
        node = self:
        for char in word:
            if char not in node.children:
                node.children[char] = Trie()
            node = node.children[char]
            node.count += 1 # 统计前缀出现次数

    def query(self, prefix):
        node = self
        for char in prefix:
            if char not in node.children: return 0
            node = node.children[char]
        return node.count
```

3.4. 树状数组 (Binary Indexed Tree, BIT)

$$\text{核心点：用}O(\log n)\text{实现单点修改和区间求和。利用}$$

$$lowbit\text{技巧：}lowbit(x) = x \ \& \ (-x)\text{。}$$

$$\text{对于数组}A\text{，树状数组}C[x]\text{管理的范围是}(x - lowbit(x), x]\text{。}$$

🚩 题目：动态数组区间和

【代码实现】

```
class BIT:
    def __init__(self, n):
        self.tree = [0] * (n + 1)

    def update(self, i, delta): # 单点修改
        while i < len(self.tree):
            self.tree[i] += delta
            i += i & (-i) # lowbit 向上修补

    def query(self, i): # 前缀和查询
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & (-i) # lowbit 向下累加
        return s
```

3.5. 线段树 (Segment Tree)

核心点：区间问题的“终极武器”。支持区间修改（Lazy Tag）和区间查询。

其逻辑基于分治。

$$\text{对于区间}[L, R]\text{，其左子树为}[L, mid]\text{，右子树为}[mid + 1, R]\text{。}$$

数学表达：

$$\text{查询区间}[l, r]\text{的和：}\sum_{i=l}^{r} A[i] = Query(root, l, r)$$

【代码实现（简化版：单点修改+区间查询）】

*注：Python 递归慢，大题建议用数组实现线段树

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (4 * self.n)
        self._build(data, 1, 0, self.n - 1)

    def _build(self, data, node, start, end):
        if start == end:
            self.tree[node] = data[start]
            return
        mid = (start + end) // 2
        self._build(data, 2 * node, start, mid)
        self._build(data, 2 * node + 1, mid + 1, end)
        self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def query(self, node, start, end, l, r):
        if r < start or end < l: return 0
        if l <= start and end <= r: return self.tree[node]
        mid = (start + end) // 2
        return self.query(2 * node, start, mid, l, r) + \
               self.query(2 * node + 1, mid + 1, end, l, r)
```

这一板块是蓝桥杯的“大题票仓”。掌握了图论与搜索，基本就拿到了省赛一等奖的入场券。

4.1. DFS 与回溯 (Depth First Search & Backtracking)

核心点：通过递归探索所有可能性。

- **排列组合/子集**：核心在于“选”或“不选”，以及递归后的“状态重置”（回溯）。
- **剪枝策略**：
  - **可行性剪枝**：当前路径已违规，立即停止。
  - **最优性剪枝**：当前代价已超过已知最小代价，立即停止。

🚩 题目：数字全排列

> **描述**：给定 n 个不同的数字，输出所有可能的排列。

【代码实现】

```
def permute(nums):
    res = []
    path = []
    used = [False] * len(nums)

    def dfs():
        if len(path) == len(nums):
            res.append(path[:]) # 记录结果
            return

        for i in range(len(nums)):
            if not used[i]:
                # 1. 做选择
                used[i] = True
                path.append(nums[i])
                # 2. 递归
                dfs()
                # 3. 撤销选择（回溯核心）
                path.pop()
                used[i] = False
    dfs()
    return res
```

4.2. BFS 与多源 BFS (Breadth First Search)

核心点：按“层”扩展。

- **最短路**：在**边权均为 1** 的图中，BFS 第一次到达终点即是最短路。
- **多源 BFS**：将所有起点同时加入队列，视为第 0 层。

🚩 题目：地图分析（距离陆地最远的海域）

> **描述**：0$代表海，1 代表陆地。找到一个海域，使其距离最近的陆地距离最大。

【代码实现】

```
from collections import deque

def max_distance(grid):
    n = len(grid)
    q = deque()
    # 1. 多源初始化：将所有陆地同时入队
    for i in range(n):
        for j in range(n):
            if grid[i][j] == 1:
                q.append((i, j))

    if not q or len(q) == n * n: return -1

    dist = -1
    while q:
        dist += 1
        for _ in range(len(q)):
            x, y = q.popleft()
            for dx, dy in [(0,1), (0,-1), (1,0), (-1,0)]:
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] == 0:
                    grid[nx][ny] = 1 # 标记为已访问
                    q.append((nx, ny))
    return dist
```

4.3. 拓扑排序 (Topological Sort)

核心点：解决有向无环图（DAG）中的依赖顺序。

- **算法**：统计入度（In-degree），将入度为 0 的点入队。

🚩 题目：课程表（判断是否能修完所有课）

【代码实现】

```
def can_finish(num_courses, prerequisites):
    indegree = [0] * num_courses
    adj = [[] for _ in range(num_courses)]
    for cur, pre in prerequisites:
        indegree[cur] += 1
        adj[pre].append(cur)

    q = deque([i for i in range(num_courses) if indegree[i] == 0])
    count = 0
    while q:
        u = q.popleft()
        count += 1
        for v in adj[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                q.append(v)
```

```
    return count == num_courses
```

4.4. 最短路径 (Dijkstra & Floyd)

核心点:

- **Dijkstra**：单源最短路（边权非负）。贪心思想 + 堆优化。时间复杂度

$$O(E \log V)$$

- **Floyd**：全源最短路。动态规划思想。时间复杂度

$$O(V^3)$$

🚩 题目：网络延迟时间 (Dijkstra 堆优化)

【代码实现】

```
import heapq

def dijkstra(start, n, adj):
    # adj: [[(v, w), ...], ...]
    dist = [float('inf')] * (n + 1)
    dist[start] = 0
    pq = [(0, start)] # (距离, 节点)

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue # 已经有更短的了，跳过

        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

4.5. 最小生成树 (Kruskal & Prim)

核心点:

- **Kruskal**：贪心思想。将所有边按权重排序，配合 **并查集 (DSU)** 判断是否形成环。

$$O(E \log E)$$

- **Prim**：加点法，适合稠密图。

🚩 题目：连接所有城市的最小费用

【代码实现 (Kruskal)】

```
def kruskal(n, edges):
    # edges: [(w, u, v), ...]
    edges.sort() # 按权值排序
    parent = list(range(n + 1))

    def find(x):
        if parent[x] != x: parent[x] = find(parent[x])
        return parent[x]

    res, count = 0, 0
    for w, u, v in edges:
        root_u, root_v = find(u), find(v)
        if root_u != root_v:
```

```
            parent[root_u] = root_v
            res += w
            count += 1
            if count == n - 1: break
    return res if count == n - 1 else -1
```

4.6. 并查集 (DSU) 应用：冗余连接

核心点：判断图中是否存在环。如果在连接 `u` 和 `v` 之前，它们的根节点已经相同，则说明这一条边会构成环。

动态规划（DP）是蓝桥杯中的“王冠明珠”，也是区分省一和省二的分水岭。其核心在于：**确定状态、推导转移方程、优化空间。**

5.1. 线性 DP (Linear DP)

核心点：状态沿着数组下标线性增长。

- $LIS(\text{最长递增子序列})$：$dp[i]$表示以$nums[i]$结尾的最长递增子序列长度。
- $LCS(\text{最长公共子序列})$：$dp[i][j]$表示$S1$前i个字符与$S2$前j个字符的最长公共子序列长度。

🚩 题目：最长公共子序列 (LCS)

【状态转移方程】

$$IfS1[i] == S2[j] : dp[i][j] = dp[i-1][j-1] + 1$$

$$Else : dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

【代码实现】

```
def lcs(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[n][m]
```

5.2. 背包九讲 (Knapsack Problems)

核心点：

- **01 背包**：每件物品仅 1 个。**逆序**遍历容量以优化空间。
- **完全背包**：每件物品无限个。**顺序**遍历容量。
- **多重背包**：每件物品 K 个。利用**二进制拆分**转化为 01 背包。

🚩 题目：01 背包（空间优化版）

> $$\text{描述：容量为}W\text{的背包，物品重量}w_i\text{，价值}v_i\text{，求最大价值。}$$

【代码实现】

```
def knapsack_01(W, weights, values):
    dp = [0] * (W + 1)
    for i in range(len(weights)):
        # 必须逆序，防止同一层循环中重复选择同一物品
        for j in range(W, weights[i] - 1, -1):
            dp[j] = max(dp[j], dp[j - weights[i]] + values[i])
    return dp[W]
```

5.3. 区间 DP (Interval DP)

核心点：状态定义为一段区间 [i, j]。通常从小区间推导到大区间。

🚩 题目：石子合并

> **描述**：N 堆石子排成一列，每次合并相邻两堆，费用为两堆重量之和。求合并为一堆的最小费用。

【状态转移方程】

$$dp[i][j] = \min_{i \le k < j} \{dp[i][k] + dp[k+1][j]\} + \sum_{x=i}^{j} weight[x]$$

【代码实现】

```
def stone_merge(n, weights):
    # 前缀和加速区间求和
    s = [0] * (n + 1)
    for i in range(n): s[i+1] = s[i] + weights[i]

    dp = [[0] * (n + 1) for _ in range(n + 1)]

    # length 是区间长度
    for length in range(2, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + s[j] - s[i-1])
    return dp[1][n]
```

5.4. 树形 DP (Tree DP)

核心点：在树结构上做 DP，通常利用 DFS 自底向上计算。

🚩 题目：没有上司的舞会 (类似打家劫舍 III)

> **描述**：每个职员有快乐值。若某人的直接上司参加，则他不能参加。求最大快乐值。

【代码实现】

```
import sys
sys.setrecursionlimit(200000)

def solve():
    # dp[u][0] 表示 u 不参加, dp[u][1] 表示 u 参加
    dp = [[0, 0] for _ in range(n + 1)]

    def dfs(u):
        dp[u][1] = happy[u]
        for v in adj[u]:
            dfs(v)
            # u 参加，儿子 v 必不能参加
            dp[u][1] += dp[v][0]
```

```
            # u 不参加，儿子 v 可参加可不参加
            dp[u][0] += max(dp[v][0], dp[v][1])

    # dfs(root)
```

5.5. 状态压缩 DP (State Compression DP)

$$\text{核心点：当}N\text{很小（通常}\leq 20\text{）时，用二进制整数表示一个集合的状态。}$$

🚩 题目：最短哈密顿路径

> **描述**：从 0 到 n-1 经过每个点恰好一次的最短路径。

【状态定义】

$$dp[state][j]\text{：当前经过的点集状态为}state\text{（二进制），当前处于点}j\text{。}$$

【代码实现】

```
def hamiltonian(n, dist):
    # dp[state][j]
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0 # 初始状态：只经过了点 0，当前在点 0

    for state in range(1, 1 << n):
        for j in range(n):
            if (state >> j) & 1: # 如果当前集合包含 j
                for k in range(n):
                    if (state >> k) & 1: # 尝试从集合中的 k 点转移过来
                        dp[state][j] = min(dp[state][j], dp[state ^ (1 << j)][k] + dist[k][j])
    return dp[(1 << n) - 1][n - 1]
```

6.1. 区间调度问题 (Interval Scheduling)

核心点：在给定的一系列区间中，选择尽可能多互不重叠的区间。

- **策略**：每次选择**结束时间最早**且与已选区间不重叠的区间。
- **证明（交换论证法）**：假设存在一个最优解 O 和贪心解 G。如果 G 的第一个区间与 O 的第一个区间不同，由于 G选择的是结束时间最早的，我们可以把 O 的第一个区间换成 G 的第一个区间，此时占用的时间更少，剩下的空间更大，至少不会比原方案差。以此类推，G 就是最优的。

🚩 题目：参加会议

> **描述**：给定 n 个会议的开始和结束时间，每个时间点只能参加一个会议，求最多能参加多少个。

【代码实现】

```
def max_meetings(meetings):
    # meetings: [(start, end), ...]
    # 核心：按结束时间排序
    meetings.sort(key=lambda x: x[1])

    count = 0
    last_end = -1
    for s, e in meetings:
        if s >= last_end:
            count += 1
            last_end = e # 更新当前占用的截止时间
    return count
```

6.2. 微观操作求最值 (Sorting + Priority Queue)

核心点：当面临动态变化的最值需求时，简单的排序已不足够，需要配合**优先队列（堆）**。通常逻辑是：先按某一维度排序，再用堆动态维护另一维度的最优性。

🚩 题目：超市利润最大化

> 描述：n件商品，每件有利润p_i和过期时间d_i。每天只能卖一件，求最大利润。

【贪心逻辑】：

1. 按过期时间从小到大排序。
2. 遍历商品，直接加入小顶堆。
3. 如果当前堆中商品数量 > 当前商品的过期时间，说明卖不完了，此时**踢出堆中利润最低**的一件。

【代码实现】

```
import heapq

def max_profit(items):
    # items: [(profit, deadline), ...]
    items.sort(key=lambda x: x[1]) # 按过期时间排序
    pq = [] # 小顶堆，存储已选商品的利润

    for p, d in items:
        heapq.heappush(pq, p)
        # 如果选的商品天数超过了截止日期，扔掉利润最薄弱的
        if len(pq) > d:
            heapq.heappop(pq)

    return sum(pq)
```

6.3. Huffman 树思想 (Huffman Coding / Merge)

> 核心点：每次合并两个权值最小的节点。这种策略能保证最终所有节点的\\路径长度 × 权值//之和最小。

- **应用场景**：文件压缩、合并果子、最小代价连接绳子。

🚩 题目：合并果子 (K-ary Huffman Tree 简化版)

> **描述**：有 n 堆果子，每次可以合并两堆，消耗的体力为两堆重量之和。求合并成一堆的最小体力。

【代码实现】

```
import heapq

def merge_fruit(fruits):
    # 1. 全部入堆
    heapq.heapify(fruits)
    total_cost = 0

    # 2. 只要还有超过一堆，就取最小的两堆合并
    while len(fruits) > 1:
        n1 = heapq.heappop(fruits)
        n2 = heapq.heappop(fruits)
        cost = n1 + n2
        total_cost += cost
        heapq.heappush(fruits, cost)

    return total_cost
```

数学与数论是蓝桥杯中必考的“填空题常客”和“大题辅助”。这些知识点通常对代码效率有极高要求，必须熟练掌握模运算下的处理技巧。

7.1. 最大公约数 (GCD) 与 最小公倍数 (LCM)

核心点:

- **GCD**：辗转相除法（欧几里得算法）。
- $$LCM：利用公式LCM(a,b) = \frac{a \times b}{GCD(a,b)}。$$
- $$性质：GCD(a,b) = GCD(b, a \bmod b)。$$

🚩 题目：等差数列

> **描述**：给定一个等差数列的一部分（无序），求包含这些数且项数最少的等差数列的项数。
>
> **思路**：项数最少意味着公差 d 必须最大，而 d 是所有相邻数差值的 **最大公约数**。

【代码实现】

```
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return (a * b) // gcd(a, b)

# Python 3.9+ 内置支持：math.gcd(*integers)
```

7.2. 快速幂 (Fast Power / Binary Exponentiation)

$$核心点：在O(\log n)时间内计算a^n \bmod m。$$

- $$原理：将n拆解为二进制，如a^{13} = a^8 \cdot a^4 \cdot a^1。$$

🚩 题目：大数幂取模

【代码实现】

```
def fast_pow(a, n, m):
    res = 1
    a %= m
    while n > 0:
        if n % 2 == 1: # 如果 n 是奇数
            res = (res * a) % m
        a = (a * a) % m # 底数翻倍
        n //= 2
    return res

# Python 竞技特权：直接使用 pow(a, n, m)
```

7.3. 素数筛法 (Sieve of Eratosthenes & Euler)

核心点:

- 埃氏筛：标记合数，复杂度$O(n \log \log n)$。
- 欧拉筛（线性筛）：每个合数只被其最小质因子筛一次，复杂度$O(n)$。

🚩 题目：求 [1, N] 之间的素数个数

【代码实现（欧拉线性筛）】

```
def get_primes(n):
    primes = []
    is_prime = [True] * (n + 1)
    for i in range(2, n + 1):
        if is_prime[i]:
            primes.append(i)
        for p in primes:
            if i * p > n: break
            is_prime[i * p] = False
            if i % p == 0: break # 核心：线性优化的关键
    return primes
```

7.4. 组合数学：排列组合与乘法逆元

核心点:

- 排列组合：$C_n^m = \frac{n!}{m!(n-m)!} \pmod{P}$。
- 乘法逆元：在模运算下，除以一个数等于乘以它的逆元。当P为质数时，由费马小定理知a的逆元为$a^{P-2} \pmod{P}$。

🚩 题目：求$C_n^m \bmod 10^9 + 7$

【代码实现】

```
MOD = 10**9 + 7

def nCr_mod(n, r, p=MOD):
    if r < 0 or r > n: return 0
    # 预处理阶乘
    fact = [1] * (n + 1)
    for i in range(2, n + 1):
        fact[i] = (fact[i-1] * i) % p

    # 公式：fact[n] * inv(fact[r]) * inv(fact[n-r])
    num = fact[n]
    den = (pow(fact[r], p-2, p) * pow(fact[n-r], p-2, p)) % p
    return (num * den) % p
```

7.5. 容斥原理 (Inclusion-Exclusion Principle)

核心点：计算多个集合的并集大小。

$$\text{公式：}|A \cup B \cup C| = |A| + |B| + |C| - (|A \cap B| + |A \cap C| + |B \cap C|) + |A \cap B \cap C|。$$

🚩 题目：能被整除的数

> $$\text{描述：给定}n\text{和}m\text{个质数}p_i\text{，求}[1, n]\text{中能被至少一个}p_i\text{整除的数的个数。}$$

【代码实现（配合位运算枚举集合）】

```
def count_divisible(n, primes):
    m = len(primes)
    res = 0
    # 枚举 2^m - 1 种集合组合
    for i in range(1, 1 << m):
        t = 1 # 当前组合的最小公倍数
        cnt = 0 # 集合中元素的个数
        for j in range(m):
            if (i >> j) & 1:
                cnt += 1
                t *= primes[j]
                if t > n: break
        else:
            # 奇数个集合加，偶数个集合减
            if cnt % 2 == 1: res += n // t
            else: res -= n // t
    return res
```

递归（Recursion）是算法设计的灵魂，它将复杂问题拆解为“更小的同类问题”。在蓝桥杯中，纯递归往往会因为**重复计算**导致超时，或因为**层数过深**导致栈溢出。因此，递归必须配合**终止条件**和**记忆化**使用。

8.1. 递归逻辑设计与终止条件

核心点：

- **递归边界（Base Case）**：必须最先写，告诉程序什么时候停止。
- **递推公式（Recursive Step）**：如何从 n 推进到 n-1。
- **栈溢出预防**：Python 默认递归深度约 1000，大题务必使用 `sys.setrecursionlimit`。

🚩 题目：斐波那契数列（递归版）

> $$\text{描述：求数列第}n\text{项，}f(n) = f(n-1) + f(n-2)。$$

【代码实现】

```
import sys
# 1. 解除递归深度限制
sys.setrecursionlimit(2000)

def fib(n):
    # 2. 终止条件（Base Case）
    if n <= 1: return n
    # 3. 递归逻辑
    return fib(n-1) + fib(n-2)
```

注：此代码在 n > 40 时会极慢，因为存在大量重复计算。

8.2. 记忆化搜索 (Memoization)

核心点：

- **原理**：用一个容器（数组或字典）记录已经计算过的结果。
- **工具**：Python 内置的 `@lru_cache` 是竞赛神器，自动实现记忆化。

🚩 题目：爬楼梯进阶版（带权值）

> **描述**：楼梯有 n 阶，每次爬 1 或 2 阶。每阶有代价 cost[i]，求爬到顶部的最小代价。

【代码实现】

```
from functools import lru_cache

cost = [10, 15, 20, 10, 30]
n = len(cost)

@lru_cache(None) # 开启记忆化，None 表示不设缓存上限
def dfs(i):
    if i >= n: return 0 # 终止条件：到达顶部

    # 递归逻辑：选择爬1阶或2阶的最小值
    res = cost[i] + min(dfs(i + 1), dfs(i + 2))
    return res

# print(dfs(0))
```

8.3. 结合排列组合：子集生成

核心点：递归处理"选或不选"的决策树。

🚩 题目：所有子集

> **描述**：给定一个集合 [1, 2, 3]，输出其所有子集（幂集）。

【代码实现】

```
def get_subsets(nums):
    res = []
    n = len(nums)

    def dfs(index, path):
        if index == n:
            res.append(path[:]) # 到达叶子节点，记录路径
            return

        # 决策1：不选当前数字
        dfs(index + 1, path)

        # 决策2：选当前数字
        path.append(nums[index])
        dfs(index + 1, path)
        path.pop() # 回溯，恢复状态

    dfs(0, [])
    return res
```

8.4. 树形问题：二叉树的最大深度

核心点：树天生就是递归结构。处理树时，通常遵循“左孩子递归、右孩子递归、最后汇总”的逻辑。

【数学表达】：

$$Height(node) = \max(Height(node.left), Height(node.right)) + 1$$

【代码实现】

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_depth(root):
    if not root: return 0 # 终止条件

    left_h = max_depth(root.left)
    right_h = max_depth(root.right)

    return max(left_h, right_h) + 1
```

9.1. 分而治之的核心三步走

1. **分解 (Divide)**：将原问题分解为若干个规模较小、相互独立、与原问题形式相同的子问题。
2. **解决 (Conquer)**：若子问题规模较小则直接解决，否则递归地解决各子问题。
3. **合并 (Combine)**：将各个子问题的解合并为原问题的解。

9.2. 典型应用一：归并排序 (Merge Sort)

核心点：先递归分解数组，再通过双指针有序地**合并**两个已排序的子序列。

数学时间复杂度：

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

🚩 题目：求数组中的逆序对

> **描述**：在数组中的两个数字，如果前面一个数字大于后面的数字，则这两个数字组成一个逆序对。输入一个数组，求出这个数组中的逆序对的总数。
>
> **分治思路**：在归并排序的“合并”阶段，如果左半部分元素 A[i] 大于右半部分元素 B[j]，则 A[i] 及其后面所有的元素都能与 B[j] 组成逆序对。

【代码实现】

```
def merge_sort_count(nums):
    if len(nums) <= 1:
        return nums, 0

    mid = len(nums) // 2
    # 分：递归处理左右两半
    left_arr, left_count = merge_sort_count(nums[:mid])
    right_arr, right_count = merge_sort_count(nums[mid:])

    # 治/合：合并有序数组并统计逆序对
    merged = []
    i = j = 0
    count = left_count + right_count

    while i < len(left_arr) and j < len(right_arr):
```

```
        if left_arr[i] <= right_arr[j]:
            merged.append(left_arr[i])
            i += 1
        else:
            merged.append(right_arr[j])
            # 核心：左边当前的数大于右边当前的数，则左边剩下的数都大于它
            count += (len(left_arr) - i)
            j += 1

    merged.extend(left_arr[i:])
    merged.extend(right_arr[j:])
    return merged, count
```

9.3. 典型应用二：快速排序 (Quick Sort)

核心点：通过一个基准值（Pivot）将数组**划分为两部分**，左边全小于基准，右边全大于基准。

关键点：快排的"合"是不需要操作的，因为当子区间都有序时，整体自然有序。

🚩 题目：第 K 个最大元素

> **描述**：给定整数数组 `nums` 和整数 `k`，请返回数组中第 `k` 个最大的元素。
>
> **分治思路（快速选择）**：在快排划分后，如果基准点的位置恰好是 n-k，则直接返回；否则只递归进入其中一边。复杂度可优化至 O(n)。

【代码实现】

```
import random

def quick_select(nums, k):
    # 选基准
    pivot = random.choice(nums)
    # 分三类
    big = [x for x in nums if x > pivot]
    equal = [x for x in nums if x == pivot]
    small = [x for x in nums if x < pivot]

    if k <= len(big):
        return quick_select(big, k) # 在大的里找
    elif k <= len(big) + len(equal):
        return pivot # 恰好在相等的里
    else:
        return quick_select(small, k - len(big) - len(equal)) # 在小的里找
```

9.4. 典型应用三：大整数乘法 (Karatsuba Algorithm)

核心点：将 n 位大整数拆分为两个 n/2 位整数。普通乘法需 4 次乘法，分治优化后仅需 3 次。

原理：

$$设X = a \cdot 10^{n/2} + b, Y = c \cdot 10^{n/2} + d$$

$$XY = ac \cdot 10^n + ((a+b)(c+d) - ac - bd) \cdot 10^{n/2} + bd$$

【代码实现（逻辑示意）】

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2
```

```
    high1, low1 = divmod(x, 10**m)
    high2, low2 = divmod(y, 10**m)

    # 递归计算三部分
    z0 = karatsuba(low1, low2)
    z1 = karatsuba(low1 + high1, low2 + high2)
    z2 = karatsuba(high1, high2)

    return z2 * 10**(2*m) + (z1 - z2 - z0) * 10**m + z0
```

二分查找（Binary Search）在蓝桥杯中不仅用于在数组中找一个数，更高端的用法是**“二分答案”**。它是解决“最优解”问题的利器，尤其是题目中出现**“最大值最小化”**或**“最小值最大化”**这类字眼时，几乎百分之百是二分答案。

10.1. 核心思维：从“找数”到“找边界”

- **普通二分**：在有序序列中找目标值。
- **二分答案**：答案在一个明确的范围内（比如 0 到 10^9），且具有**单调性**（如果 x 满足条件，那么所有 >x 的值也一定满足/不满足）。我们不直接算答案，而是二分猜测一个答案，然后写一个 `check` 函数判断这个猜测是否合法。

10.2. 典型应用一：基础二分查找（边界处理）

核心点：避免死循环，处理 `l` 和 `r` 的更新。

🚩 题目：数的范围

> **描述**：给定一个升序数组，求目标值 `target` 出现的起始位置和终止位置。

【代码实现】

```
def search_range(nums, target):
    if not nums: return [-1, -1]

    # 查找左边界
    l, r = 0, len(nums) - 1
    while l < r:
        mid = (l + r) // 2
        if nums[mid] >= target: r = mid
        else: l = mid + 1
    if nums[l] != target: return [-1, -1]
    start = l

    # 查找右边界
    l, r = 0, len(nums) - 1
    while l < r:
        mid = (l + r + 1) // 2 # 注意这里 +1 避免死循环
        if nums[mid] <= target: l = mid
        else: r = mid - 1
    end = l
    return [start, end]
```

10.3. 典型应用二：二分答案（最大值最小化）

核心点：将求解问题转化为**判定问题**。

🚩 题目：切绳子 / 木棒切割

> **描述**：有 N 根长度不一的绳子，现在要切出 M 根长度相同的绳子，求这 M根绳子最大长度是多少？

【二分逻辑】：

1. **区间**：长度最小是 0，最大是绳子中最长的那根。
2. **单调性**：切出的长度越短，得到的段数越多；长度越长，段数越少。
3. **Check函数**：给定长度 x，计算所有绳子能切出多少段

$$\text{看是否} \geq M$$

【代码实现】

```
def check(mid, lengths, m):
    count = 0
    for l in lengths:
        count += l // mid
    return count >= m

def solve():
    n, m = map(int, input().split())
    lengths = list(map(int, input().split()))

    l, r = 1, max(lengths)
    ans = 0
    while l <= r:
        mid = (l + r) // 2
        if mid == 0: break
        if check(mid, lengths, m):
            ans = mid  # 这个长度可以，尝试更长的
            l = mid + 1
        else:
            r = mid - 1 # 太长了，缩短点
    print(ans)
```

10.4. 典型应用三：最小值最大化（进击的奶牛）

核心点：在固定资源下，让最小的间隔尽可能大。

🚩 题目：进击的奶牛 / 摊位安排

> **描述**：N 个坐标点，安置 C头牛，要求相邻两头牛之间的最小距离最大。

【代码实现】

```
def check(distance, positions, c):
    count = 1 # 第一头牛放第一个点
    last_pos = positions[0]
    for i in range(1, len(positions)):
        if positions[i] - last_pos >= distance:
            count += 1
            last_pos = positions[i]
    return count >= c

def solve():
    positions.sort()
    l, r = 0, positions[-1] - positions[0]
    res = 0
    while l <= r:
        mid = (l + r) // 2
        if check(mid, positions, c):
            res = mid
            l = mid + 1 # 尝试增大间隔
        else:
            r = mid - 1 # 间隔太大，放不下 C 头牛
    return res
```

15.1. KMP 算法 (Knuth-Morris-Pratt)

核心点：当子串与主串不匹配时，利用已经匹配过的部分信息，将子串向后“跳跃”尽可能远的距离，而不是从头开始。

- **Next 数组（前缀函数）**：`next[i]` 表示子串 $0 \sim i$ 这一段中，**相等的前缀和后缀**的最大长度。

🚩 题目：子串出现的位置

> **描述**：给定主串 S和模式串 P，求 P 在 S 中所有出现的起始下标。

【代码实现】

```
def get_next(p):
    nxt = [0] * len(p)
    j = 0
    for i in range(1, len(p)):
        while j > 0 and p[i] != p[j]:
            j = nxt[j - 1]
        if p[i] == p[j]:
            j += 1
        nxt[i] = j
    return nxt

def kmp(s, p):
    nxt = get_next(p)
    res = []
    j = 0
    for i in range(len(s)):
        while j > 0 and s[i] != p[j]:
            j = nxt[j - 1]
        if s[i] == p[j]:
            j += 1
        if j == len(p):
            res.append(i - len(p) + 1)
            j = nxt[j - 1]
    return res
```

15.2. 字符串哈希 (String Hashing)

核心点：将字符串看作一个 **P 进制数**，通过取模映射为一个数值。

- 公式：$H(S) = (s_1 \cdot P^{n-1} + s_2 \cdot P^{n-2} + \cdots + s_n \cdot P^0) \pmod M$
- 区间哈希值：利用前缀哈希数组$h[i]$，可以在$O(1)$时间内求出任意子串$[l, r]$的哈希值。

$$h[l \ldots r] = (h[r] - h[l-1] \cdot P^{r-l+1}) \pmod M$$

- $**$常用参数$**$：$P = 131$或13331，$M = 2^{64}$（$Python$中直接用大数或取模$10^9 + 7$）。

🚩 题目：字符串判等

> **描述**：给定一个长字符串，多次询问其中两个子串 [l1, r1]和 [l2, r2]是否完全相同。

【代码实现】

```
P, MOD = 131, 10**18 + 7 # 使用大质数防止碰撞

def string_hash(s):
    n = len(s)
    h = [0] * (n + 1)
    p = [1] * (n + 1)
    for i in range(1, n + 1):
        h[i] = (h[i-1] * P + ord(s[i-1])) % MOD
        p[i] = (p[i-1] * P) % MOD
```

```
    def get_sub(l, r): # 获取 s[l-1...r-1] 的哈希值
        return (h[r] - h[l-1] * p[r-l+1]) % MOD

    return get_sub
```

15.3. 字典树 Trie (Prefix Tree)

核心点：利用字符串的公共前缀来减少查询时间。

- **应用**：前缀统计、字符串检索、最大异或对（01-Trie）。

🚩 题目：前缀数量统计

> **描述**：插入 N 个字符串，查询以某个字符串为前缀的已插入字符串个数。

【代码实现】

```
class Trie:
    def __init__(self):
        self.son = {}  # 存储子节点
        self.cnt = 0   # 以当前节点结尾的单词数
        self.pre = 0   # 经过当前节点的单词数（即前缀计数）

    def insert(self, s):
        node = self
        for char in s:
            if char not in node.son:
                node.son[char] = Trie()
            node = node.son[char]
            node.pre += 1
        node.cnt += 1

    def query_prefix(self, s):
        node = self
        for char in s:
            if char not in node.son: return 0
            node = node.son[char]
        return node.pre
```